

s2.01 — Multi-tenant certificate authority and certificate issuance service

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Rotation state machine and static stability — IC3 baseline](#)
- [Revocation, key custody, and tenancy — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: this prompt is a JD-fit supplement in [the bank](#).
 - JD (`../0.req/oracle senior ic3.pdf`): PKI team, "certificate lifecycle management, key management, trust services, cryptographic infrastructure, secure multi-tenant cloud deployments, backward compatibility."
 - This is the hiring team's home domain, so it is the design most likely to come from the hiring manager.
 - The architecture below is a proposed interview answer.
 - Where it matches OCI Certificates it is labeled "documented OCI behavior"
 - Everything else is proposed design.
 - Cross-reference: `../4.1o-arch.md` §2 (control plane vs data plane), §3 (OCI Certificates), §5 (TLS fundamentals).
- Why this is hard
 - Naive: `openssl` on a box, certificates in a bucket, a wiki page saying "renew yearly", private keys on disk.
 - First break: expiry is a time bomb with 100% probability
 - A forgotten renewal is a hard outage of every dependent service (canonical pattern).
 - Second break: key custody
 - Any copy of a CA private key outside an HSM is a compromise story, and a leaked CA key means re-issuing the world.
 - Third break: the service cannot force consumers to switch certificates
 - Rotation only works if two versions overlap and consumers hot-reload.
 - Minimum primitives
 - HSM-held CA keys
 - Sign inside the boundary.
 - Never export plaintext CA private keys.
 - Versioned certificates with overlapping validity

- Consumers that cache and hot-reload
 - An append-only audit trail.
- What the user wants: "TLS just works"
 - Rotation never wakes me
 - Compromise is containable."
- 🗨️ "This is a control-plane correctness problem with a data-plane survival constraint: low QPS, but every mistake is either an outage-by-timer or a security incident."
- Opening script
 - "I will treat this as a control-plane certificate lifecycle service over HSM-held keys, with a data plane that must keep serving TLS while the control plane is down."
 - "First I confirm internal versus public trust, scale, who consumes the certificates, and whether revocation and audit are hard requirements."
 - "Then I show the numbers are small, so the time goes to rotation, key custody, and failure containment rather than throughput."
 - "The architecture is an async work-request API, an issuance worker that signs inside the HSM, a rotation scheduler, and a CRL publisher to object storage."
 - "The deep dive is the version state machine with an overlap window and adoption check, and why the data plane is statically stable on cached bundles."
 - "I will close with the failure table, tenant isolation, and where the design changes for short-lived per-workload certificates."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Write and read rates, HSM budget; API semantics
Architecture	17:00–30:00	Diagram, control/data plane split, one issuance trace
Deep dive	30:00–45:00	Rotation state machine, static stability, custody
Operations and tradeoffs	45:00–50:00	Failure table, revocation, tenancy, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Internal mTLS or public trust?	Internal-first (service identities); public endpoints via an external CA later	Private CA means no external rate limits and custom policy; public trust would add CA/Browser Forum rules and ACME
Scale?	1M leaf certificates, 10k consuming services, leaf TTL 90 d, CA TTL 5 y	Fixes renewal rate and shows this is not a throughput problem
Who consumes the certificates?	Load balancers, gateways, sidecars; pull-and-cache model	Consumers must cache locally so handshakes never depend on this service
Is revocation required?	Yes; CRL primary, OCSP later	Adds a signed-artifact publisher and a freshness SLO
Compliance and audit?	Every issue, revoke, rotate audited; keys in FIPS 140-2 Level 3 HSMs	Forces append-only audit and HSM-only custody

Question	Assumed answer	Why it changes the design
Who holds the leaf private key?	Requester (CSR flow) by default; service-managed key in the tenant's vault as an option	Decides whether the service ever touches private material
Tenancy model?	OCI-style compartments; one tenant never sees another's resources	Drives per-tenant CAs, quotas, and 404 -not- 403 isolation

- Functional scope
 - Create a CA (root or subordinate) whose key is generated inside an HSM.
 - Issue a leaf certificate from a CSR or a managed key specification.
 - Renew and rotate with versioning and an overlap window.
 - Revoke a version and publish a signed CRL.
 - Serve the certificate, chain, and trust bundle by version with an ETag.
 - List certificates expiring before a date for operational sweeps.

Requirement	Target	Enforced by
Issue latency	p99 < 2 s end to end; HSM sign 10-50 ms (assumed)	Async work request, queue, HSM connection pool
Read (get bundle)	p99 < 50 ms, 99.99%	Read replicas, ETag, CDN-able bundle endpoint
Control-plane availability	99.9% for issue/rotate/revoke	Stateless API fleet across 3 fault domains; replicated metadata store
Data-plane resilience	Zero handshake failures during a 24 h control-plane outage	Consumers cache bundle on disk; renewal happens at 2/3 of lifetime
Rotation	Zero downtime; overlap ≥ 7 d between versions	Version state machine; deprecation only after adoption check
Consistency	1 committed current version per certificate	Expected-base CAS, current attempt fence, and issuer-scoped serial uniqueness
Durability	Certificates and audit immutable for 7 y	Append-only audit table; WORM archive in object storage
Security	0 plaintext CA private-key exports; 100% durable mutation audit intent	HSM custody for CA keys; separately scoped managed leaf-key delivery
CRL freshness	nextUpdate ≤ 24 h; rebuild within 15 min of a revoke	CRL builder on revoke event plus daily timer; age alarm

- Invariants
 - A private CA key exists only inside an HSM
 - The service stores a key handle, never key bytes.
 - Every issued version names exactly one issuing CA version and one serial unique within that CA.
 - A version that is CURRENT has at least one overlapping valid predecessor until adoption is confirmed.
 - A revoked version never returns to CURRENT.
 - Every mutating operation writes an audit row in the same transaction as the state change.
 - Tenant A's credentials cannot observe the existence of tenant B's resources.
- Non-goals
 - Public WebPKI trust and ACME
 - Buy that from a public CA.
 - Code-signing and end-user client certificates in the first release.
 - Building workload identity
 - Reuse the platform's resource principal or SPIFFE identity.
 - Designing the HSM/KMS service itself
 - That is [s2.04](#).

Capacity and consequence — IC3 baseline

- Writes (issuance and renewal)
 - 1M leaves with a 90 d TTL renewed at 2/3 of life means each leaf renews every 60 d.
 - $1M / 60 \text{ d} \approx 16.7k \text{ renewals/day} \approx 0.2/s$ average
 - Tiny.
 - Bursts happen when a CA rotates or a fleet is re-imaged
 - Fan-out can be 10k+ in an hour.
 - Decision: issuance is queued and rate-capped per CA, never synchronous, so a burst becomes a backlog rather than a timeout.
- HSM signing budget
 - Asymmetric Sign on a managed HSM is order of magnitude hundreds per second per region (check the current quota page).
 - Forcing re-issue of 1M leaves after a CA rotation at 100/s takes $10,000 \text{ s} \approx 2.8 \text{ h}$ of saturated HSM.
 - Decision: CA rotation does not re-issue leaves
 - The new CA version signs at the next scheduled renewal and the old chain stays valid through the overlap.
 - Decision: renewal times carry jitter so fleets do not renew in lockstep.
- Reads (bundle fetch)
 - 10k services fetching once per hour is $10,000 / 3,600 \approx 3/s$
 - A fleet restart can spike to thousands per second.
 - Decision: the bundle endpoint is immutable per version with an ETag and long cache headers, so spikes hit the cache, not the metadata store.
- CRL size
 - Revoked is $\ll 1\%$
 - 10k revoked serials at roughly 40 B each is about 400 KB signed.
 - Decision: publish the CRL to object storage behind a CDN and re-sign it on revoke and at least every 24 h
 - Verifiers never fetch it from the API fleet.
- Storage
 - 1M certificates with about 3 retained versions and 6 KB of PEM chain each is about 18 GB
 - Small.
 - Audit at roughly 20k events/day and 1 KB per event for 7 y is about 50 GB.
 - Decision: keep hot metadata in one relational store per region
 - Archive audit to WORM object storage after 90 d.
- So what
 - This is not a scale problem
 - Say so in the first two minutes.
 - Spend the time on rotation, key custody, and failure containment.

Architecture and flow — IC3 baseline

```
[control plane]
Client/CI/Terraform -> API gateway (authn, compartment policy, quota)
                        -> Certificate API service -> metadata store (CAs, CA versions,
                        |                               certificates, versions, work requests, audit)
                        | issue/rotate/revoke = work request (202 + work_request_id)
                        v
[async]  Issuance worker -> Vault/HSM: Sign(key_ref, TBS) -> store version PENDING -> CURRENT -> event
[async]  Rotation scheduler -> finds versions at 2/3 lifetime -> enqueue renew (idempotent per version)
[async]  CRL builder -> re-sign CRL on revoke + every 24 h -> object storage bucket -> CDN

[data plane / hot path]
Consumers (LB, gateway, sidecar) -> GET bundle by version (ETag, long cache) -> disk cache
    TLS handshakes use the cached chain; they never call the control plane
```

- API gateway owns authentication, compartment policy, and quota.
 - Mechanism bought: one policy check point (allow group X to manage certificates in compartment Y) and a per-tenant token bucket.
 - Cost paid: every request pays a policy lookup
 - Policy-service latency is on the control-plane path.
 - Certificate API service owns validation and intent.
 - Mechanism bought: validate synchronously, persist intent, return 202 with a work-request id ([s3.01](#)).
 - Cost paid: every client needs a poll loop
 - A fast operation still looks asynchronous.
 - Metadata store owns the source of truth for CAs, versions, and audit.
 - Mechanism bought: one relational store per region gives transactions across version flip and audit row.
 - Cost paid: single-region writes
 - A region loss stops issuance until DR ([s2.06](#)).
 - Issuance worker owns signing.
 - Mechanism bought: Sign executes inside the HSM under an audited key handle
 - The worker sees only public output.
 - Cost paid: HSM latency and quota are on the issuance path, and an HSM signature cannot be rolled back by a database transaction.
 - Rotation scheduler owns renewals.
 - Mechanism bought: a periodic sweep at 2/3 of lifetime with idempotent enqueue.
 - Cost paid: a scheduler bug is a fleet-wide expiry
 - It needs its own "expiring soon" alarm as a backstop.
 - CRL builder owns revocation publication.
 - Mechanism bought: a signed, versioned CRL artifact on object storage behind a CDN.
 - Cost paid: revocation is only as fresh as the publish interval and the verifier's cache.
 - Consumers own the data plane.
 - Mechanism bought: local disk cache plus hot-reload gives static stability during control-plane outages.
 - Cost paid: the service cannot force a switch
 - Adoption must be measured from consumer telemetry.
- Client sends `POST /v1/certs` with `Idempotency-Key`, CSR, SANs, and TTL.
- Gateway authenticates the caller and checks compartment policy and quota.
- API verifies the CSR self-signature (proof of possession) and checks SANs and TTL against the tenant's profile.
- API persists the certificate row, a `PENDING` version with a reserved serial, a work request, and an audit row in one transaction.
- API returns 202 with `work_request_id`.
- Issuance worker locks the certificate row, builds the TBS certificate, and calls `Sign(key_ref, SHA-256(TBS))` in the HSM.

- Worker stores the signed PEM chain, flips the version `PENDING` → `CURRENT`, marks the prior version `PREVIOUS`, and writes the audit row.
- Worker publishes `CertRotated`; consumers refresh on event or on their next poll and pick up the new ETag.
- Client polls the work request until `SUCCEEDED`, then reads `GET /v1/certs/{id}/bundle`.

API and data model — IC3 baseline

Surface	Contract	Idempotency	Conflict/error behavior
<code>POST /v1/cas</code>	<code>{name, type: ROOT\ SUBORDINATE, parent_ca_id?, key_algo, ttl};</code> returns <code>202 {ca_id, work_request_id}</code>	Idempotency-Key; same key and payload returns the same work request	Same key with different payload returns <code>409</code> ; root creation needs a separate administrative policy
<code>POST /v1/cas/{id}:rotate</code>	Creates a new CA signing version; returns <code>202 {work_request_id}</code>	Idempotent per current CA version	<code>409</code> if a rotation for this CA is already in flight
<code>POST /v1/certs</code>	<code>{ca_id, csr or managed_key_spec, san[], ttl_days, auto_renew};</code> returns <code>202 {cert_id, work_request_id}</code>	Idempotency-Key per tenant	<code>400</code> on bad CSR signature or disallowed SAN; <code>429</code> on tenant quota
<code>GET /v1/certs/{id}</code>	<code>CURRENT</code> version metadata, issuer, <code>not_after</code> , state	Read	<code>404</code> for another tenant's id (no existence leak)
<code>GET /v1/certs/{id}/bundle</code>	PEM chain of the current version; ETag; <code>?version=</code> for an explicit version	Read	<code>304</code> on <code>If-None-Match</code> ; immutable per version
<code>POST /v1/certs/{id}:renew</code>	Expected base version; returns <code>202</code> with work request	Same tenant/cert/base/token and payload returns the original operation	<code>409</code> only for conflicting payload or base version
<code>POST /v1/certs/{id}:revoke</code>	<code>{version, reason};</code> returns <code>202</code> ; lands in the next CRL	Idempotent (already revoked returns the same result)	<code>409</code> if the version is <code>PENDING</code> and never issued; revoke of <code>CURRENT</code> requires an explicit flag
<code>GET /v1/cas/{id}/crl</code>	Signed CRL for the CA version (also on CDN)	Read	Redirect to the object-storage CDP
<code>GET /v1/certs?expiring_before=&after=</code>	Cursor page for operational sweeps	Read	Indexed on (<code>tenant_id</code> , <code>not_after</code>)
<code>GET /v1/work-requests/{id}</code>	<code>ACCEPTED\ IN_PROGRESS\ SUCCEEDED\ FAILED</code> , percent complete, errors	Read	Authoritative answer after a lost <code>202</code>

- Path note
 - The API companion spells these as `/certificate-authorities` and `/certificates`
 - The short names above are interview shorthand for the same contracts.
- Versioning and backward compatibility (JD keyword)
 - Additive fields only
 - `/v1` is frozen.
 - Deprecation ships with `Sunset` headers and a documented window.
 - Old consumers that read `bundle` without `?version=` keep receiving the `CURRENT` version.

State	Key	Fields/invariant
CA	<code>(tenant_id, ca_id)</code>	Parent, state <code>CREATING\ ACTIVE\ DISABLED\ DELETED</code> , current signing version, profile, optimistic version
CA signing version	<code>(tenant_id, ca_id, issuer_version)</code>	HSM <code>key_ref</code> , public chain, state <code>PENDING\ CURRENT\ PREVIOUS\ DEPRECATED\ REVOKED</code> , validity
Certificate	<code>(tenant_id, cert_id)</code>	Owner, name, <code>auto_renew</code> , <code>current_version</code> ; single writer via row lock
Certificate version	<code>(tenant_id, cert_id, version_num)</code>	Issuer version, serial unique per CA, PEM chain (public only), state, validity, revocation
Work request	<code>(tenant_id, work_request_id)</code>	Operation, status, resource id, idempotency key, errors
Audit	<code>id</code> append-only	Actor, action, resource, detail; no <code>UPDATE/DELETE</code> grants
Idempotency record	<code>(tenant_id, idempotency_key)</code>	Request hash and result id; retained $\geq$ the client retry horizon

- Schema constraints
 - Use composite tenant ownership keys for every parent/child relationship.
 - Persist the complete immutable issuance intent before the first signature request.

```

cas PRIMARY KEY (tenant_id, ca_id)
  parent reference -> cas(tenant_id, parent_ca_id)
ca_versions PRIMARY KEY (tenant_id, ca_id, issuer_version)
  FOREIGN KEY (tenant_id, ca_id) -> cas
certificates PRIMARY KEY (tenant_id, cert_id)
  UNIQUE (tenant_id, cert_id, ca_id)
  FOREIGN KEY (tenant_id, ca_id) -> cas
cert_versions PRIMARY KEY (tenant_id, cert_id, version_num)
  FOREIGN KEY (tenant_id, cert_id, ca_id) -> certificates
  FOREIGN KEY (tenant_id, ca_id, issuer_version) -> ca_versions
  UNIQUE (tenant_id, ca_id, serial)
intent: base_version, serial, issuer_version, TBS_DER, TBS_hash,
      policy_generation, state, attempt_fence, lease_until
result: certificate_DER, chain_hash, not_before, not_after, revoked_at
renewal_intents UNIQUE (tenant_id, cert_id, base_version)
      operation_id, state, retry_deadline
outbox PRIMARY KEY (operation_id, transition_version)
  inserted in the same transaction as the corresponding state transition

```

- The schema is pseudocode for ownership and uniqueness constraints.
 - Generate executable DDL only after selecting the actual database and deletion policy.
- Reference DDL (simplified; the companion file holds the full schema)

```

CREATE TABLE cas (
  tenant_id    uuid NOT NULL,
  ca_id        uuid NOT NULL,
  parent_ca_id uuid,                -- NULL = root
  name         text NOT NULL,
  key_ref      text NOT NULL,       -- HSM key handle, NEVER key material
  state        text NOT NULL,       -- CREATING|ACTIVE|DISABLED|DELETED
  current_issuer_version int NOT NULL DEFAULT 0,
  version      bigint NOT NULL DEFAULT 0, -- optimistic lock; serializes issuance per CA
  PRIMARY KEY (tenant_id, ca_id),
  FOREIGN KEY (tenant_id, parent_ca_id) REFERENCES cas (tenant_id, ca_id)
);

CREATE TABLE certificates (
  tenant_id    uuid NOT NULL,
  cert_id      uuid NOT NULL,
  ca_id        uuid NOT NULL,
  name         text NOT NULL,
  auto_renew   boolean NOT NULL DEFAULT true,
  current_version int,
  PRIMARY KEY (tenant_id, cert_id),
  FOREIGN KEY (tenant_id, ca_id) REFERENCES cas (tenant_id, ca_id)
);

CREATE TABLE cert_versions (
  tenant_id    uuid NOT NULL,
  cert_id      uuid NOT NULL,
  version_num  int NOT NULL,
  issuer_version int NOT NULL,
  state        text NOT NULL,       -- PENDING|CURRENT|PREVIOUS|DEPRECATED|REVOKED|FAILED
  serial       numeric(49) NOT NULL, -- X.509 serial; positive, <= 20 octets, enforced in code
  tbs_hash     bytea NOT NULL,      -- pinned issuance intent
  attempt_fence bigint NOT NULL DEFAULT 0,
  lease_until  timestamptz,
  pem_chain    text,                -- public material only; NULL while PENDING
  not_before   timestamptz,
  not_after    timestamptz,
  revoked_at   timestamptz,
  revocation_reason text,
  PRIMARY KEY (tenant_id, cert_id, version_num),
  FOREIGN KEY (tenant_id, cert_id) REFERENCES certificates (tenant_id, cert_id)
);

CREATE UNIQUE INDEX cv_serial ON cert_versions (tenant_id, cert_id, serial);
CREATE INDEX cv_expiring ON cert_versions (tenant_id, not_after) WHERE state = 'CURRENT';

CREATE TABLE audit_log (
  id          bigserial PRIMARY KEY, -- append-only; no UPDATE/DELETE grants
  tenant_id   uuid, actor text, action text,
  resource    text, detail jsonb,
  at          timestamptz NOT NULL DEFAULT now()
);

```

- The serial unique index above is scoped per certificate for brevity
 - The ownership rule in the pseudocode schema scopes it per issuer namespace (`tenant_id, ca_id, serial`), which is the constraint to state in the interview.
- Schema notes
 - Serial uniqueness follows the issuer-name namespace in X.509.
 - This design conservatively shares one allocation space across all versions of a tenant's logical CA.
 - CA key rotation preserves that allocation space.
 - Independent CA objects reusing an issuer name need a shared serial allocator or a prohibition on that name reuse.
 - The X.509 rule (positive, at most 20 octets) is enforced in the issuer code
 - SQL precision alone is insufficient.

- `pem_chain` is nullable so a `PENDING` row can exist before the HSM signs
- That row is the durable issuance intent.
- Work-request rows follow the [s3.01](#) shape and are omitted here.

Rotation state machine and static stability — IC3 baseline

```
issue v(n+1):  PENDING --verified sign + fenced publication--> CURRENT      (v(n) -> PREVIOUS)
               PENDING --HSM sign fails after retries--> FAILED
overlap window (>= 7 d): v(n+1) CURRENT and v(n) PREVIOUS both validate
deprecate:    PREVIOUS --after window AND adoption check--> DEPRECATED (still parseable; alarm if seen)
revoke:       CURRENT|PREVIOUS --revoke--> REVOKED (terminal; lands in next CRL)
```

- Renewal timing
 - Renew at 2/3 of lifetime: a 90 d certificate renews on day 60.
 - That leaves 30 d to detect and fix consumers that did not switch.
 - The mechanism reference ([s5.01](#) C.5) uses 50% for 24 h certificates
 - The fraction is a knob, the principle is "renew well before expiry, with jitter".
 - The OCI rule fields are `renewal_interval` and `advance_renewal_period`.
 - P90D and P30D are this design's assumed values.
 - Imported certificates and externally managed private-key flows require the supported manual renewal path.
- Adoption check before deprecating
 - Data-plane telemetry answers "which certificate serial did handshakes use in the last 24 h?"
 - Do not deprecate `PREVIOUS` while its serial still serves traffic.
 - Without telemetry, deprecation is a guess
 - That guess is the classic rotation outage.
- CA rotation is the same machine one level up, plus cross-signing
 - Create a new HSM key and a new CA signing version (`PENDING`).
 - Cross-sign: the old CA version signs a certificate for the new CA's public key.
 - Cross-signing can preserve a path to the old anchor only when client path builders and certificate constraints accept that path.
 - Distribute the new chain to trust stores first
 - Switch the signer only after coverage is high enough.
 - Restrict the old key to required CRL signing after issuer cutover.
 - Retain that capability through the required final post-expiry scheduled CRL publication.
 - Full treatment: [s2.02](#).
- OCI contract boundaries

Topic	Documented OCI behavior	Proposed design in this file
Version stages	<code>CURRENT</code> , <code>PENDING</code> , <code>LATEST</code> , <code>PREVIOUS</code> , <code>DEPRECATED</code> , <code>FAILED</code> , <code>PENDING_ACTIVATION</code> (externally managed CA)	Same names minus <code>LATEST</code> and <code>PENDING_ACTIVATION</code> ; adds <code>REVOKED</code> as a terminal state
Deprecated versions	A <code>DEPRECATED</code> version cannot be made current again	Same; <code>DEPRECATED</code> is one-way and only reached after the adoption check
Revocation	Tracked as <code>revocation_status</code> on a version; revoking a CA version is supported for subordinate CAs only	<code>REVOKED</code> state plus <code>revoked_at</code> and reason; root revocation is a trust-store change, not an API call
Renewal	Creates another version with new contents and validity; <code>renewal_interval</code> and <code>advance_renewal_period</code> are ISO 8601 durations	Scheduler sweep at 2/3 of lifetime with jitter; same two knobs

Topic	Documented OCI behavior	Proposed design in this file
Old versions	Not deleted automatically; service limits cap versions per certificate	Retain PREVIOUS through the overlap; garbage-collect DEPRECATED after audit retention
CA key custody	Requires an HSM-protected asymmetric Vault key; software-protected keys are not supported; RSA 2048/4096 or ECDSA NIST_P384	Same; the service stores only key_ref
Hierarchy	Path length constraint 0-10 for subordinate CAs; mixed key families in one chain are not supported	Two-tier by default; one key family per chain
CRL	Published to a dedicated Object Storage bucket per CA; CDP URL may substitute {issuing_ca_version_number}	Same bucket-plus-CDN model; one CRL object per CA signing version

- Renewal scheduling flow
 - Select due certificates through the expiry index with bounded batches.
 - Under the certificate lock, compare the expected current version.
 - Upsert one durable renewal intent for that base version and its dispatch outbox.
 - Resume an existing incomplete intent instead of treating every existing row as complete.
 - A queue deduplication key reduces redundant delivery but is not the correctness boundary.

```
SCHEDULE_RENEWAL(tenant, certificate, expectedBase):
    transaction:
        lock certificate; require current_version == expectedBase
        intent = insert_or_read_unique(tenant, certificate, expectedBase)
        if intent newly inserted: insert dispatch outbox
        return intent.operation_id
```

```
def tick_renewals(db, queue, now):
    """Rotation scheduler sweep: idempotent per (cert, base version)."""
    rows = db.query("""
        SELECT c.tenant_id, c.cert_id, cv.version_num, cv.not_before, cv.not_after
        FROM certificates c JOIN cert_versions cv
        ON cv.tenant_id = c.tenant_id AND cv.cert_id = c.cert_id
        AND cv.version_num = c.current_version
        WHERE c.auto_renew AND cv.state = 'CURRENT'
        AND %s >= cv.not_before + (cv.not_after - cv.not_before) * 2 / 3
        ORDER BY cv.not_after LIMIT 1000""", (now,))
    for r in rows:
        with db.transaction():
            db.lock_certificate(r.tenant_id, r.cert_id)
            if db.current_version(r.tenant_id, r.cert_id) != r.version_num:
                continue # someone already flipped it
            intent, created = db.upsert_renewal_intent(r.tenant_id, r.cert_id, r.version_num)
            if created:
                db.insert_outbox(intent.operation_id) # same transaction as the intent
            queue.send(dedup_id=f"renew:{r.cert_id}:{r.version_num}",
                      body={"operation_id": intent.operation_id})
    # Correctness boundary: the UNIQUE (tenant_id, cert_id, base_version) intent row.
    # The queue dedup_id only reduces duplicate delivery.
```

- A failed intent remains visible with a retry or operator-resolution state.
 - Do not let a failed successor silently exclude the certificate from every future sweep.
- Issuance flow
 - Reserve one immutable intent.
 - Claim an attempt.
 - Sign the pinned TBS.
 - Verify and assemble the certificate.
 - Publish conditionally with audit and outbox.

```

ISSUE(intent_id):
    transaction:
        lock intent and certificate
        if intent committed: return recorded result
        require intent runnable and no live competing attempt
        persist tenant, base_version, issuer_key_version, serial, TBS_DER,
            TBS_hash, validity, policy_generation before first sign
        increment attempt_fence; persist lease_until
    signer.authorize_and_sign(intent_id, pinned_key_version, TBS_DER, attempt_fence)
    verify signature against pinned issuer public key
    assemble X.509 Certificate(TBSCertificate, algorithmIdentifier, signatureValue)
    transaction:
        lock intent and certificate
        require current attempt_fence and live lease
        require certificate.current_version == intent.base_version
        require intent still publishable and issuer/policy permit release
        persist certificate DER and chain
        mark old current PREVIOUS; mark new version CURRENT
        commit result, audit intent, and notification outbox together

```

```

// Issuance worker (Java 17): two short transactions around one HSM call.
public void issue(UUID tenantId, UUID intentId) {
    Pinned p = tx.run(() -> {
        Intent it = repo.lockIntentAndCertificate(tenantId, intentId); // SELECT ... FOR UPDATE
        if (it.committed()) return null; // idempotent replay
        it.requireRunnableAndNoLiveAttempt();
        long fence = repo.incrementAttemptFence(it, Duration.ofSeconds(30)); // lease_until
        return new Pinned(it.tbsDer(), it.issuerKeyVersion(), it.serial(), fence);
    });
    if (p == null) return;
    byte[] sig = hsm.sign(p.issuerKeyVersion(), p.tbsDer(), p.fence()); // signs inside the HSM, audited
    X509Certificate cert = X509.assembleAndVerify(p.tbsDer(), sig, issuerPublicKey(p));
    tx.run(() -> {
        Intent it = repo.lockIntentAndCertificate(tenantId, intentId);
        it.requireFence(p.fence()); // stale attempt cannot publish
        it.requireBaseVersionStillCurrent();
        repo.persistVersion(it, cert); // PENDING -> CURRENT, old ->
        PREVIOUS
        audit.log("cert.issue", it, Map.of("serial", p.serial()));
        outbox.enqueue(new CertRotated(it.certId(), it.newVersion())); // consumers refresh on event or
        poll
    });
}
// A crash after hsm.sign leaves a signed but unpublished certificate for a tracked serial;
// the retry re-signs the same TBS and the fence decides which attempt publishes.

```

- Crash after signing
 - Resume the same serial and exact TBS bytes.
 - Randomized signatures may differ while identifying the same tracked certificate intent.
 - A superseded or cancelled signed intent is quarantined and revoked when required.
 - A database fence prevents stale publication.
 - The signing broker must independently enforce authorization and writer ownership before physical signing.
 - A SQL rollback cannot undo a signature already produced.
- Consistency boundary
 - The `PENDING` row in transaction 1 is the durable intent
 - The flip in transaction 2 is the commit.
 - Signing the same TBS twice yields the same serial, so a retry after a crash cannot mint an untracked certificate.
 - Keep database transactions short around the HSM call.
 - No database transaction alone makes a physical signature atomic with metadata publication.
- Static stability: the data plane survives a control-plane outage

- Consumers cache the full chain and key locally on disk and refresh on interval or event with the ETag.
- Handshakes can continue without this service while cached keys, the full chain, and required revocation evidence remain valid.
- Corollary interviewers probe: cache on boot.
 - A consumer that fetches on startup with no local copy dies during a control-plane outage.
 - Ship last-known-good in the deploy artifact or persist the cache to disk.
- Alarm on "certificates expiring in < 14 d"
- During a long outage this is the countdown clock.
- Chaos test: isolate consumers for 24 h after verifying at least that much remaining certificate and status validity.
- Separately test near-expiry material and expect the profile's declared failure behavior.
- Consumer hot-reload
 - Watch a file or event, rebuild the `SSLContext`, and swap an `AtomicReference` read by a delegating `X509ExtendedKeyManager`.
 - Envoy uses SDS pushes
 - Nginx uses `reload`.
 - Existing sessions keep the old certificate until they close
 - New handshakes use the new one.
 - Never require a process restart
 - Restart storms are self-inflicted outages.

Revocation, key custody, and tenancy — IC3 baseline

- Revocation: CRL freshness versus availability
 - Revoke is a state flip plus audit plus a CRL rebuild
 - The CRL is signed, versioned, and carries `nextUpdate ≤ 24 h`.
 - Relying-party policy chooses soft-fail or hard-fail for unavailable status.
 - A valid signed revoked result is not an availability error and must not become soft-fail acceptance.
 - Short certificate lifetimes bound residual exposure for clients that do not enforce fresh revocation status.
 - Choose lifetime by the controlled workload's renewal capability, not a claimed universal public-web value.
 - OCSP with stapling is the follow-up: the server staples a signed freshness proof, which removes the client fetch but adds responder availability as a dependency.
 - Documented OCI behavior: the CRL is published to a dedicated Object Storage bucket configured on the CA, with an optional custom HTTPS CDP URL
 - OCSP is not described in the overview.
 - Full treatment: [s2.03](#).
- Key custody
 - CA keys: HSM only, referenced by handle
 - `sign` is an HSM operation that returns a signature, never a key.
 - Leaf keys are requester-held in the CSR flow.
 - A managed exportable leaf key needs encrypted storage and an authenticated delivery boundary.
 - A nonexportable leaf key instead requires a compatible remote-signing TLS integration.
 - The CA signing key never leaves the HSM in plaintext.
 - Managed leaf-key generation and delivery require a separate custody review.
 - Documented OCI behavior: HSM-protected key versions cannot be exported
 - The public key can be downloaded
 - The HSMs are FIPS 140-2 Level 3.
 - Proof of possession: verify the CSR self-signature before policy checks so denials do not leak which names exist.
 - Possessing a key proves nothing about authorization for a DNS name

- SANs, key usage, and validity are authorized from policy independently of the CSR.
- Serial numbers
 - Proposed serials use 128 random bits with a positive, at-most-20-octet encoding and an issuer-scoped uniqueness check.
 - Random serials are the chosen design.
 - Sequential serials are not universally forbidden by X.509
 - Public-trust policy requirements are a separate scope.
- Multi-tenancy and authorization (JD keyword)
 - Every resource lives in a tenant compartment
 - Policy is checked at the gateway with OCI-style statements.
 - Quotas: certificates per tenant and issue rate per tenant through a token bucket ([s1.01](#)).
 - Cross-tenant isolation test: tenant A's token must get 404, not 403, on tenant B's certificate ids so existence never leaks.
 - Defense in depth: one subordinate CA per tenant with X.509 Name Constraints for that tenant's names, so a policy bug in tenant A's rules still cannot produce a certificate for tenant B's names.
 - Full treatment: [s2.05](#).

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Consumer never adopts the new version	One service; outage when PREVIOUS expires	Adoption telemetry shows old serial still serving; expiring-in-14 d alarm	PREVIOUS stays valid through the overlap	Fix the consumer's reload; extend overlap; never deprecate while the old serial serves traffic
Control plane or metadata store down	All new issue/rotate/revoke in the region	Health checks; work-request backlog age	Handshakes continue on cached bundles; renewals queue	Fail over the store replica; scheduler catches up on the backlog; countdown is the nearest not_after
HSM or Vault unavailable or throttled	New signatures fail; running services unaffected	HSM error and 429 rate; issuance success rate	Work requests stay IN_PROGRESS; retries with jitter; per-tenant 429 with Retry-After	Raise quota; spread across per-environment keys; renewals were scheduled with 30 d of slack
Crash between HSM sign and version flip	One certificate version	Reconciler finds PENDING row older than a threshold	Version stays PENDING; CURRENT unchanged	Re-sign the same TBS (same serial) or mark FAILED and CRL the serial
Stale CRL (builder or publish failure)	Verifiers accept revoked certificates (soft-fail) or reject everything (hard-fail)	CRL age gauge; alarm when nextUpdate - now < 2× publish interval	Old CRL keeps being served from the CDN	Restart the builder; re-sign; short TTLs bound the exposure
Subordinate CA key compromise	Its issued leaves and identities	Anomalous signing and custody evidence	Stop affected signing; strict verifiers may reject the chain	Revoke through the parent and reissue through an uncompromised issuer; verify client adoption
Root CA key compromise	Entire trust hierarchy	Unexpected root signing or custody breach	Stop affected signing and treat the anchor as untrusted	Distribute a new independently trusted root out of band; do not rely on cross-signing by the compromised root
Rotation scheduler stalls or double-runs	Expiry risk or duplicate delivery	Expiry inventory and intent age	Durable renewal identity prevents a second logical renewal	Resume incomplete intents and catch up within HSM quotas

Failure	Blast radius	Detection	Degraded behavior	Recovery
Audit write fails	Compliance gap, not a serving outage	Audit write error rate; page immediately	Mutating operations fail closed (audit is in the same transaction)	Restore the audit store; reads and handshakes never depended on it
Clock skew on a consumer	One host reports "not yet valid"	notBefore rejections in client logs	Back-date notBefore by 5 min	NTP fix; monitoring on drift

- Observe
 - Issuance success rate and HSM `sign` latency.
 - Expiring-in-N-days count
 - Page at 14 d.
 - CRL age and `nextUpdate - now`.
 - Rotation adoption lag (time from `CURRENT` flip to last old-serial handshake).
 - Work-request backlog age.
 - Audit write failures (page immediately).
 - Per-tenant quota rejections.
- Retry discipline
 - Exponential backoff with jitter on HSM and store transients.
 - Bounded attempts and a total deadline per work request
 - After that the version becomes `FAILED`, never silently retried forever.
 - Retryable
 - HSM `5xx` and throttling
 - Store transients
 - CRL upload.
 - Terminal
 - Policy violation
 - Bad CSR signature
 - Access denied
 - Key disabled.
 - Retries preserve the serial and exact TBS intent.
 - They can produce multiple signature attempts, each attributable to that known intent.
- Tenant isolation
 - Every CA, certificate, and CRL bucket is scoped to a compartment
 - Policy is evaluated per request at the gateway.
 - Quotas cap certificates and issue rate per tenant so one tenant's burst cannot exhaust the shared HSM budget.
 - A tenant's token receives `404` on another tenant's ids, not `403`, to avoid existence leaks.
 - Per-tenant subordinate CAs with Name Constraints bound what a policy bug can sign.
- Rollout and rollback safety
 - Canary the API fleet by fault domain
 - The data plane never depends on API health, so a bad deploy pauses issuance and nothing else.
 - Schema changes follow expand, migrate, contract
 - A new worker must accept `PENDING` rows written by the previous version.
 - Version-state transitions are monotonic, so rolling back the worker cannot un-flip a `CURRENT` version.
 - Chaos test: isolate consumers for 24 h after verifying at least that much remaining certificate and status validity.
 - Separately test near-expiry material and expect the profile's declared failure behavior.
- Runbook: expired certificate incident
 - Issue an emergency short-TTL certificate from the subordinate CA.
 - Hot-reload the consumer.

- Root-cause why the adoption alarm and the 14 d alarm did not fire.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Two-tier hierarchy (chosen)	Root authorizes subordinate issuers	Separates root custody from routine signing	More path and lifecycle management	Baseline multi-tenant PKI with bounded issuer blast radius
Single root	One CA issues everything	Simplest	Root key on the issuance path; compromise means re-issue the world	Dev and test only
Automatic versioned renewal (chosen)	Early renewal with measured adoption	Reduces missed expiry	Requires key/CSR automation and telemetry	Managed keys and consumers capable of automated renewal
CRL primary (chosen)	Signed list on object storage and CDN	No online dependency; cacheable	Freshness bounded by publish interval; soft-fail clients	Internal fleets and LB integrations
OCSP plus stapling	Server presents a fresh signed status	Per-certificate freshness; no client fetch	Responder availability becomes a dependency	Browser-facing public certificates
Async work request (chosen)	202 plus poll for every mutation	Keeps HSM latency and serialization off the sync path; uniform for Terraform	Every client needs a poll loop	Control-plane resources
Synchronous issuance	201 with the certificate inline	Simple for interactive flows	Timeouts under HSM throttling; retries duplicate work	Add as a fast lane only for interactive developer flows
Managed CA (OCI Certificates, AWS Private CA)	Buy the signer and versioning	Native compartment authz, audit, LB integration	Limited policy expressiveness; no ACME	The OCI fleet; say it is the team's own product
SPIFFE/SPIRE workload identity	Per-node agents issue short-lived SVIDs	Identity and certificates together; rotation built in	Agent on every node; no long-lived certificates	Service-mesh mTLS at scale; pair with a managed CA upstream

- Defeaters to volunteer
 - "Store the CA key encrypted in the database."
 - ⇒ Encrypted-at-rest is not custody; the key exists in service memory during every signature.
 - Sign inside the HSM; the service holds a key handle, never key bytes.
 - "Rotation is an atomic swap of the certificate."
 - ⇒ Consumers cache; without overlap every cached client breaks at once.
 - Two valid versions overlap for at least 7 d; deprecate only after the adoption check.
 - "Rely on revocation to contain a compromise."
 - ⇒ Verifiers soft-fail; a revoked certificate keeps working wherever the CRL is unreachable or cached.
 - Short TTL is the honest control; revocation is the backstop.
- IC4 stretch — stronger requirement: a certificate per pod
 - Short-TTL (24 h) certificates, no reliance on revocation, SPIFFE-style identity.
 - Issuance rises 100x, so now it is a scale problem: HSM pool, per-CA sharding, or an intermediate signing tier.
 - Delegate a short-lived, name-constrained sub-CA per node or cluster
 - The HSM signs one sub-CA per node per day instead of one leaf per pod.
 - Consumer-side renewal at 50% of lifetime so the CA can be down for half a TTL before any expiry.
- IC4 stretch — 10x load
 - Reads: bundle endpoint stays immutable per version behind the CDN
 - 10x reads never reach the store.

- Writes: shard the metadata store by tenant with the certificate row as the unit of serialization
 - Avoid locking one CA row for every issuance by reserving serials per certificate row under the (ca_id, serial) constraint.
- HSM: partition tenants across HSM partitions or keys so one tenant's burst cannot starve another's renewals.
- IC4 stretch — cost and blast radius
 - Cell the control plane per region and per availability-domain group
 - A store failure strands one cell's issuance, not the region.
 - Per-tenant subordinate CAs mean a CA compromise re-issues one tenant, not the fleet.
 - Cost drivers are HSM operations and retained versions, not storage
 - Garbage-collect DEPRECATED versions after the audit horizon.
- IC4 stretch — migration and BYOCA
 - For an external issuer, generate the subordinate key in the selected HSM and submit its CSR for external signing.
 - Import the resulting CA certificate chain through the supported externally issued CA workflow.
 - Importing a root PEM alone does not give the service signing authority.
 - Legacy consumers (an Oracle DB client that reads a wallet and trusts RSA only) get an rsa-legacy profile on its own subordinate with a sunset date, so its longer TTLs never widen the ECDSA fleet's blast radius.
- Weakest assumption
 - Consumers reliably report which serial served each handshake
 - Without that telemetry the adoption check is a guess.
 - Mitigation: treat missing telemetry as "not adopted", extend the overlap, and alarm on the countdown to PREVIOUS expiry.
- Over-engineering warning
 - Do not build OCSP, multi-region active-active CAs, or a custom KMS for a 0.2/s internal issuance service.
 - Do not build workload identity
 - Consume the platform's resource principal or SPIFFE identity.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Internal versus public trust, who consumes, revocation and audit as hard requirements	Names non-goals (WebPKI, identity, KMS) and says why buying them is cheaper	Scopes the PKI instead of designing all of PKI
Estimates	0.2/s writes, 3/s reads, CRL in KBs	HSM Sign budget forces "CA rotation does not re-issue leaves" and renewal jitter	A number changes a decision, not just a slide
Architecture	Control plane versus data plane split; async work requests; HSM handle only	States the cost of each component and why the store is single-region per cell	Every box has a mechanism and a price
Deep dive	Version state machine with overlap and adoption check; static stability on cached bundles	Two-transaction issuance around the HSM sign and the orphan-serial bound; cross-signed CA rotation	Reasons across the database and HSM boundary
Operations	Failure table, expiring-in- 14 d alarm, CRL age, 404 -not- 403	Per-tenant subordinate CAs with Name Constraints; cells; rollback that cannot un-flip a version	Volunteers containment before being asked
Wrap	Recap, overlap tradeoff	Names adoption telemetry as the weakest assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Where are the private keys?
 - **IC3 baseline**

- CA signing keys remain in the HSM.
 - CSR leaf keys remain with their requester.
 - Managed exportable leaf keys need a separate encrypted storage and authenticated delivery boundary.
- **IC4 stretch**
 - Documented OCI behavior requires an HSM-protected Vault key for a CA and rejects software-protected keys
 - Verify the CSR self-signature before policy so denials do not leak names.
- Q2: The root CA is compromised.
 - Now what?
 - **IC3 baseline**
 - Root compromise requires a new independently trusted anchor and out-of-band trust-store distribution.
 - Cross-signing by the compromised root cannot restore authenticity.
 - **IC4 stretch**
 - Alarm on any root `Sign` outside a ceremony window
 - The failure is always a verifier you did not know existed, so build the trust-store inventory before the rotation, not during.
- Q3 (premise reversal): Why not use Let's Encrypt for everything and skip this service?
 - **IC3 baseline**
 - Public trust for public endpoints, yes
 - Internal mTLS wants a private CA with no external rate limits, internal names, custom policy, and no dependency on an external issuer for service identity.
 - **IC4 stretch**
 - The honest build case on OCI is policy and inventory on top of the managed Certificates service
 - The signer and versioning should be bought, and saying so is the IC4 signal.
- Q4: Now give every pod its own certificate.
 - **IC3 baseline**
 - Short-TTL (24 h) certificates with no revocation reliance and SPIFFE-style identity
 - Issuance rises 100x, so it becomes a scale problem for the HSM pool.
 - **IC4 stretch**
 - Delegated name-constrained sub-CAs per node move the HSM off the hot path
 - The HSM signs one sub-CA per node per day.
- Q5: How do consumers hot-reload without a restart?
 - **IC3 baseline**
 - Watch a file or event, rebuild the `SSLContext`, swap an `AtomicReference` behind a delegating `KeyManager`
 - Envoy SDS or Nginx reload
 - Existing sessions finish on the old certificate.
 - **IC4 stretch**
 - The consumer reports the serving serial back as telemetry, which is what makes the adoption check possible.
- Q6: How do you guarantee serial uniqueness across a CA key rotation?
 - **IC3 baseline**
 - This design chooses 128 random serial bits with issuer-scoped uniqueness.
 - Sequential serials are not universally prohibited by X.509.
 - **IC4 stretch**
 - All signing versions of one logical CA share one serial namespace
 - Enforce the positive, at-most-20-octet rule in the issuer because SQL precision alone does not.
- Q7: A database transaction cannot roll back an HSM signature.
 - Where is your consistency boundary?
 - **IC3 baseline**

- Reserve the intent (serial, validity, TBS hash) in one transaction, sign, then flip in a second transaction
 - A crash between leaves a `PENDING` row whose serial is known and revocable.
- **IC4 stretch**
 - Re-signing the same TBS yields the same serial, so retries never mint an untracked certificate
 - The accepted residual is one revocable serial bounded by TTL, and you say that bound out loud.
- Q8: Disabling a CA races an issuance in flight.
 - **IC3 baseline**
 - Recheck the CA state at the signing boundary under the row lock
 - Issuance after `DISABLED` fails the work request, and disabling issuance is not revoking existing certificates.
 - **IC4 stretch**
 - Document the cutoff explicitly (work requests accepted before the disable complete or fail, never half-sign), and avoid making the CA row a global hot row by reserving serials per certificate.
- Validation scenarios
 - Renewal at day `60` with the old serial still serving on day `67`
 - Deprecation must wait.
 - Isolate consumers for 24 h with > 24 h remaining chain and required status validity.
 - Verify declared failure behavior separately with near-expiry certificates and stale status.
 - Consumer restarted with an empty disk cache during the outage.
 - Worker crash between HSM sign and version flip
 - Reconciler outcome.
 - Double-run of the rotation scheduler
 - No duplicate versions.
 - Revoke followed by CRL builder failure
 - CRL age alarm fires before `nextUpdate`.
 - Tenant A token on tenant B certificate id returns `404`.
 - Subordinate CA rotation with cross-sign
 - Old-trust-store verifier still validates.
 - HSM throttling during a fleet re-image
 - Renewals back off without duplicating work.

Recall card

- "Control-plane correctness with a data-plane survival constraint: low QPS, every mistake is an outage-by-timer or a security incident."
- "Two valid versions overlap; consumers switch; deprecate only after the adoption check."
- "Issuance is an HSM signing operation against a key handle, never a key export."
- "Handshakes never call the control plane; cached bundles are the static stability."
- "Revocation enforcement depends on the relying party and the lifetime of cached evidence."
- 5 numbers: `90 d` leaf TTL renewed at day `60`
 - Overlap $\geq 7 d$
 - CRL `nextUpdate` $\leq 24 h$
 - Issue p99 `2 s`
 - Page at `14 d` to expiry.
- Failures to rehearse
 - Consumer non-adoption.
 - Control-plane outage near expiry.
 - Root compromise requiring an independent trust path.
- 3 defeaters: encrypted-DB key custody
 - Atomic-swap rotation

- Revocation as the primary control.
- Done when: rotation state machine drawn from memory
 - Static-stability answer in `30 s`
 - Schema written cold.
- Wrap-up script
 - “The design is an async control plane over HSM-held keys, versioned certificates with an overlap window, a CRL published to object storage, and consumers that cache bundles so the data plane survives a control-plane outage.”
 - “The key invariant is that a version is never deprecated while its serial still serves traffic, and every issuance reserves a known serial before the HSM signs.”
 - “The main tradeoff is async issuance with overlap: every client needs a poll loop and we retain two valid versions, in exchange for zero-downtime rotation and HSM latency off the request path.”
 - “My weakest assumption is consumer adoption telemetry; without it the deprecation step is a guess, so I would validate that first and extend the overlap when data is missing.”

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [PKI and KMS mechanism reference](#): X.509 anatomy, chain validation, CRL/OCSP, HSM signing, CA design doc, rotation flows.
- [Async control-plane resource API](#): work-request contract used by every mutation here.
- Sibling designs
 - [s2.02 renewal and CA rollover](#)
 - [s2.03 revocation](#)
 - [s2.04 HSM signing](#)
 - [s2.05 authorization](#)
 - [s2.06 regional DR](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values, TTLs, overlap windows, and the `2/3` renewal fraction remain proposed interview assumptions.
- [OCI Certificates overview](#): CA hierarchy, root versus subordinate, imported CAs, CRL, IAM and Load Balancer integration, no mixed key families in a chain.
- [OCI creating a certificate authority](#)
 - HSM-protected Vault key required
 - RSA `2048/4096` or ECDSA `NIST_P384`
 - Path length `0-10`
 - CRL bucket and CDP URL with `{issuing_ca_version_number}`.
- [OCI making a certificate version current](#): a `DEPRECATED` version cannot be made current.
- [OCI renewing a certificate](#): renewal creates a new version
 - Imported and externally keyed certificates cannot auto-renew
 - Old versions are not deleted automatically.
- [OCI Certificates Management SDK](#), `VersionStage`
 - Stage names `CURRENT`
 - `PENDING`
 - `LATEST`
 - `PREVIOUS`
 - `DEPRECATED`

- FAILED
- PENDING_ACTIVATION.
- [OCI Certificates Management SDK](#), [CertificateRenewalRule](#)
 - `renewal_interval` and `advance_renewal_period` as ISO 8601 durations.
- [OCI Vault key overview](#): HSM-protected keys cannot be exported, FIPS 140-2 Level 3, key versions on rotation, RSA and ECDSA signing.
- [OCI IAM policies](#): compartment-scoped policy statements used for tenant isolation.
- [RFC 5280](#): certificate and CRL profile
 - Serial number rules
 - Name Constraints.
- [RFC 6960](#): OCSP, cited for the stapling follow-up.
- [Let's Encrypt DST Root CA X3 expiration](#): the canonical cross-sign and root-rotation story.
- SPIFFE/SPIRE and AWS Private CA/ACM documentation: analogy sources for workload identity and managed-CA rotation.